

1. O resultado da execução do programa será [1], [1, 2]. O motivo é que a lista que representa o argumento com valor padrão é inicializada apenas uma vez — especificamente, quando a função é definida, e não a cada chamada. Assim, a lista ficará vazia na primeira chamada; então, o elemento 1 será adicionado a ela. Portanto, na segunda chamada, a lista terá o elemento 1 e, após a adição do elemento 2, resultará em [1, 2]. O conceito usado aqui é o de parâmetros em Python com valores padrão mutáveis.

2. O resultado produzido pela execução do programa é 0 5. O motivo é que a linguagem Java usa a passagem de parâmetros por valor, mas, para objetos, há duas etapas: a referência do objeto é copiada e, em seguida, a referência ao objeto é armazenada na nova variável. Como resultado, as alterações feitas na variável `v` serão aplicadas ao array original. Quanto à variável inteira `n`, ela é um tipo primitivo Java, portanto, seu valor é copiado para a variável no método. Assim, para a variável `n`, 0 será atribuído apenas na variável local do método, enquanto a variável no bloco principal continuará contendo 5. O conceito usado aqui é a passagem de parâmetros por valor em Java.

3. A resposta esperada à execução do programa é [2, 2, 2]. Esse resultado ocorre devido à linguagem Python permitir fechamentos. Nesse contexto, a palavra-chave `map` cria fechamentos para as funções anônimas, e as três funções criadas têm acesso ao mesmo índice `i`. Como tal, ao final do loop `for`, a variável `i` terá o valor 2, e as três funções imprimirão 2 três vezes. O conceito usado aqui é o de fechamento.

4. A resposta imprimida será 3. O motivo é que a variável `n` é declarada como `static`, o que significa que ela armazena dados em memória para o programa inteiro, e não para nenhum subprograma específico. Portanto, durante a primeira chamada à função, a variável `n` aumenta de 0 para 1, mudando de 1 para 2 na segunda chamada. Finalmente, a terceira chamada, feita pelo `printf`, atualizará a variável de 2 para 3, resultando em 3 imprimido pela função `printf`. O conceito usado aqui é o de variável local estática.

5. O programa não será compilado. A razão é que, na linguagem Rust, um Vetor não pode ser passado para a função dobrar, já que o programa tentaria usar a variável `v` novamente. Isso ocorre porque a posse (Ownership) da variável `v` é transferida para a função dobrar, de modo que, quando o código chegar à linha `print! ("{}", v)`, a variável `v` não poderá mais ser encontrada. O conceito usado aqui é a posse (Ownership).

6. O programa gerará `UnboundLocalError`. A razão para esse resultado é que o código contém uma variável global chamada `total`, mas o Python não permite que variáveis locais e globais compartilhem o mesmo nome. O motivo é que, dentro da função `void`, a variável local `total` tem prioridade sobre a variável global `total`. Devido a essa prioridade, a variável local `total` não é reconhecida pela linguagem Python na expressão `total = total + x`, causando assim um erro. O conceito usado aqui é o de escopo.